Experiment No. 6

Title: Face Generation using Generative Adversarial Networks (DCGAN) on the CelebA Dataset

Aim: To implement a Deep Convolutional Generative Adversarial Network (DCGAN) using PyTorch, train it on the CelebFaces Attributes (CelebA) dataset, and generate new, realistic synthetic human face images using the trained Generator, with an optional deployment as a Flask web application.

Objectives:

1. To understand the architecture and working principle of a Generative Adversarial Network (GAN), including the roles of the Generator and Discriminator.
2. To implement the Generator and Discriminator networks using convolutional and transposed-convolutional layers (DCGAN architecture) in PyTorch.
3. To train the DCGAN on the CelebA dataset using adversarial (Binary Cross-Entropy) loss and the Adam optimizer.
4. To visualize training progress through loss curves and generated image grids, and to generate new, previously unseen human face images using the trained Generator.

Pre-Requisites:

1. Basic knowledge of Python programming.
2. Basic understanding of Deep Learning, Convolutional Neural Networks (CNNs) and PyTorch.
3. Basic understanding of probability and adversarial (min-max) optimization concepts.
4. Basic familiarity with the CelebA dataset and image preprocessing.
5. Basic knowledge of Google Colab and GPU execution is desirable.
6. Basic knowledge of Flask for the optional deployment extension.

Theory:

Generative Adversarial Network: A Generative Adversarial Network (GAN) is a Generative AI model consisting of two neural networks — a Generator and a Discriminator — trained simultaneously through adversarial competition. The Generator learns to produce realistic synthetic data, while the Discriminator learns to distinguish between real and generated (fake) data.

DCGAN Architecture: The DCGAN used in this experiment replaces fully-connected layers with convolutional layers. The Generator uses a series of transposed-convolution (ConvTranspose2d) layers with Batch Normalization and ReLU activations to upsample a latent noise vector into a 64×64 RGB face image, ending with a Tanh activation. The Discriminator uses convolution (Conv2d) layers with Batch Normalization and LeakyReLU

activations to downsample an image into a single probability score, ending with a Sigmoid activation.

Latent Noise Vector: The Generator's input is a random latent vector z, typically of dimension 100, sampled from a standard normal distribution N(0, I). Each different noise vector, when passed through the trained Generator, produces a different synthetic face image.

Adversarial (Min-Max) Loss: The GAN is trained using the min-max objective min_G max_D E[log D(x)] + E[log(1 − D(G(z)))]. The Discriminator is trained to maximize its ability to correctly classify real images (x) as real and generated images (G(z)) as fake. The Generator is trained to maximize the probability that the Discriminator classifies its generated images as real, using Binary Cross-Entropy loss for both networks.

Generation using the Generator: After training, new face images can be generated without any input image. A random latent vector z is sampled from N(0, I) and passed directly to the trained Generator, which converts it into a new, realistic-looking human face that did not exist in the original dataset.

Tools and Technologies Required

1. Google Colab
2. Python 3
3. PyTorch and Torchvision
4. NumPy
5. Matplotlib
6. CelebA dataset (via Kaggle)
7. GPU support in Google Colab (recommended)
8. Flask, HTML/CSS and JavaScript for the optional deployment extension

Procedure:

1. Open Google Colab and select a GPU runtime.
2. Import PyTorch, Torchvision, NumPy and Matplotlib libraries.
3. Download the CelebA dataset using the Kaggle API (or mount Google Drive as an alternative).
4. Unzip the dataset and locate the aligned face images inside the img_align_celeba folder.
5. Display a few sample CelebA images to understand the input data.
6. Define image preprocessing transforms: resize to 64×64, center-crop, convert to tensor, and normalize pixel values to the range [−1, 1].
7. Build a custom CelebADataset class to load face images directly from the folder.

8. Create a DataLoader with a batch size of 128 to feed images during training.
9. Visualize a batch of real training images from the DataLoader.
10. Define the Generator network using ConvTranspose2d, BatchNorm2d and ReLU layers, ending with a Tanh output layer.
11. Define the Discriminator network using Conv2d, BatchNorm2d and LeakyReLU layers, ending with a Sigmoid output layer.
12. Apply custom weight initialization (Normal distribution, mean = 0, std = 0.02) to both networks, as recommended by the DCGAN paper.
13. Define the Binary Cross-Entropy loss function and two separate Adam optimizers (one each for the Generator and Discriminator).
14. Create a fixed noise vector to track the Generator's progress on the same inputs across training epochs.
15. Train the Discriminator on real and fake image batches, then train the Generator to fool the Discriminator, repeating this adversarial process for multiple epochs.
16. Log and plot the Generator and Discriminator losses across training iterations.
17. Save a grid of generated images at the end of every epoch to visually track the Generator's progress.
18. Save Generator and Discriminator model checkpoints periodically during training.
19. Generate new face images by passing random noise vectors to the trained Generator.
20. Generate and save a single new face image from the trained Generator.
21. Save the final trained Generator model (generator.pth) for later deployment.
22. Build a simple Flask web application with a “Generate Face” button that loads the trained Generator and displays newly generated faces in the browser.

Conclusion:

The experiment successfully demonstrated the implementation of a Deep Convolutional Generative Adversarial Network (DCGAN) on the CelebA face dataset using PyTorch. Students learned how the Generator and Discriminator compete against each other in an adversarial training process, how convolutional and transposed-convolutional layers are used to build stable GAN architectures, and how a trained Generator can synthesize new, realistic human face images from random latent noise vectors. Training and validation losses were visualized to observe the adversarial learning dynamics. The saved Generator model also provided the foundation for developing a Flask-based face-generation web application with a simple “Generate Face” button.